

CAPSTONE PROJECT REPORT

Virtual - LocalRobots2



Submitted by:

Logan Huber ,Rose Marie Godoy, Alec Pizziferro, Steven Watanabe, Josephine Qualls, Pranaav
Jamigumpala

Under the guidance of/Submitted to:

Dr. Tony Grichnik

Bradley University, Department of Computer Science

Table of Contents

1. Abstract	3
Overview	3
Objectives	3
2. What is Webots?	3
About Webots	3
Our Use Case	3
Webots & DOGZILLA Interoperability	4
3. Attempted Solutions	4
Problems with ROS	4
Differences in Previous Year's Robot	5
Gait Solver Attempts	5
4. Final Implementation	6
Process	6
API	6
5. Future Work	7
Improvements to Workflow and Simulation	7
Spring Semester	8
6. Resources and Directories	8
Github	8
7. References	8

1. Abstract

Overview

We have created a simulation that will allow us to experiment with the functionality of the DOGZILLA robot so that we can minimize the risk of damaging the DOGZILLA robot in the real world. We have set up and iterated on an accurate representation of a quadruped robot with Webots for the purpose of simulation. Once we are satisfied with the testing conducted within the simulation, we will attempt to apply those same/similar functions to the physical robot.

Objectives

The API would ideally include all the functionalities of the physical DOGZILLA robot, so that we may simulate various possible actions the robot may perform in a controlled yet physically-accurate environment. Careful consideration of how to simulate the rotation of the legs for movement (mainly inverse kinematics), is essential to being able to get the robot to move in space properly.

2. What is Webots?

About Webots

Webots is an open-source robotics simulator designed to accurately mimic the environment found with physical robots. Using Webots, we can design and code robots before ever physically building them, all within one piece of software. We can also write controllers in various languages, allowing for portability between different robot systems.

Our Use Case

In our case, we use Webots so that it is possible for multiple people to iterate on challenges presented, without having to wait for access to a physical robot. This also allows multiple people to work on a single challenge collaboratively a bit more easily. The main goal of the virtual side of this project is to take a simulated result provided by Webots, transfer the logic over to DOGZILLA, and have a similar output. To model DOGZILLA within Webots, we have used the provided Webots representation of Boston Dynamics's Spot. Like DOGZILLA, Spot is also a quadruped dog robot, with 12 individually articulated motors.

Webots & DOGZILLA Interoperability

The main difference between Spot and DOGZILLA is that Spot is significantly larger than DOGZILLA. This difference can be accounted for by either remaking Spot to closer match DOGZILLA, or scaling the world up around Spot and translating any math accordingly. Both Spot and DOGZILLA work by applying a series of joint rotations to match a pose, using these poses to traverse. The exact methods DOGZILLA uses are unknown. For Spot to move within Webots, a custom solver is needed to calculate the poses and rotations needed. Webots does not provide a solver out of the box.

3. Attempted Solutions

Problems with ROS

Robot Operating System, or ROS is a middleware for interacting with physical robots. It is designed to make it easier to virtualize robots. DOGZILLA can use this, it even provides a guide for working with it (the documentation is a bit limited with this however). It is possible to have a simulated robot in Webots also use this, communicating in the same way that a physical robot would. This would allow Webots users access to things like RViz and other navigation tools. There is also potential interoperability between DOGZILLA and the simulated robot. It may even be possible to link the virtual robot to the physical robot simulation, which would make debugging and writing new behaviors easier.

There are a few issues with using ROS, especially for a class full of people. The main problem is the environment setup, it is very involved and requires a machine running Ubuntu or a similar Debian-based operating system. Setup typically requires getting a ROS2 workspace running and then running a lot of commands every time Webots are needed.

From our testing, it is semi-possible to get this running on Windows, but even then the feature set and stability provided did not compare to its Linux counterpart. A VM on more powerful hardware could be used, but on lower-end hardware, it would likely be too slow due to Webots needing to do 3D rendering, robot simulation, and compiling.

If, for whatever reason, there is a desire to shift to using ROS over Webots for future endeavors, it's important to note that the official website for ROS (<https://www.ros.org>) does not currently offer a functioning guide to set up ROS or ROS2 for Windows 11. Attempts to use the Windows 10 guide to set up ROS2 on Windows 11 have resulted in various issues, including some technical issues that have made it impossible to use the command prompt to utilize ROS2, along with other basic Windows functions accessed from the command prompt. While it is possible that at least part of the issues related to installing ROS2 could be caused by human error, the guide provided by the website gives little information about how to properly install the necessary components, making it suboptimal for setting up ROS2 regardless.

A more comprehensive and effective guide for setting up ROS2 was located here:

<https://github.com/LucaVits/LearningROSWithPythonTurtle/blob/main/ROS2/WindowsInstallation/ROS2IronInstallationWindows.ipynb>

While much of the information in this guide is very similar, if not equivalent, to what is available in the guide from the official ROS website, it provides additional clarification about setting up environmental variables that isn't mentioned in the official website and provides some updated links to ensure nothing outdated/unusable is inadvertently installed. An issue to note with both guides is that neither installation of OpenSSL properly serves the necessary function for utilizing ROS2. The ideal version of OpenSSL to be installed through Chocolatey has been determined to be OpenSSL 1.1.1.1400. Keep in mind, we were unable to get everything properly working with ROS2 for the necessary purpose of simulating the robot. Ideally, accommodations would be made to utilize the Linux counterpart over Windows, if there's a desire to invest time in using ROS.

Differences in Previous Year's Robot

A majority of the time spent this semester was dedicated to finding an optimal way to get a quadruped robot walking around in Webots. The main idea was simple: Webots allow us to control every joint individually, either setting its rotation or velocity, and by leveraging that we could move our robot in any direction with enough math. That math was the primary challenge of our implementation, and we took a few approaches to get it correct.

For simpler robots, such as the robot used last year, it was possible to just set a target velocity or position for each motor, which would allow the robot to move intuitively. This was not the case for the quadruped robot. We first attempted to get the robot moving by alternating pre-defined poses (similar to keyframes in animation), but we were unable to get this to function properly. There were simply too many states to account for and blend between. Determining all of these motions, while also ensuring that they moved similarly to DOGZILLA also proved to be very time-intensive.

Gait Solver Attempts

Following this attempt, one of our team members who had a large amount of experience with 3D software figured that inverse kinematics (IK) could be used for the virtual robot. IK is a technique to rotate a series of joints in a chain such that the end reaches some target position. IK is different from forward kinematics (FK) in that its motions are better suited towards linear motion, rather than angular motion, hence its usefulness for creating a gait. Further research on this revealed that DOGZILLA and other similar quadruped robots also use IK to drive their motion. Efforts were made to contact the DOGZILLA manufacturer, Yahboom, regarding how the kinematics library worked, as well as a potential Webots implementation, but they did not provide any information.

Webots provides an open-source quadruped robot simulation for a robot called Bioloid by Robotis. Bioloid is similar to DOGZILLA, but its limbs actuate in a much different way. The controller that was provided with Bioloid was converted to work with Spot, but it resulted in imprecise and limited movement. The controller essentially only allowed forward and backward movement and a fixed speed. The controller was also difficult to control from a scripting perspective, as it was written in C and designed as a technical demonstration. Bioloid's controller was converted to C++ to allow easier interoperability with Python, but it did not end up working for our use case.

Following these attempts, some research for open-source implementations of a Spot-specific controller was done. From that research, we found a repository implementing Spot via ROS2 and Webots, which led to the discovery of the aforementioned issues surrounding ROS. The repository contains a lot of useful features, which will likely be implemented in the future.

4. Final Implementation

Process

The end goal of this project is to provide an environment where students can complete challenges virtually, and then be able to translate that code to a physical robot and have it behave in the same way. We reached out to the creators of the previously mentioned Webots Spot repository, and they referred us to another repository of theirs. This repository contained a simpler, non-ROS-based version of the Spot implementation. We have forked this repository and modified it to suit our needs. The API has been modified to better match that of DOGZILLA. There may be some future optimizations if there are ever any efforts to port this code to DOGZILLA itself. The final implementation contains an IK solver for all four legs of Spot, as well as a gait trajectory solver, which uses a bezier curve to produce smooth movements. Spot can move in various directions, as well as having the beginning steps to allow Spot to traverse vertically in scenarios like stairs.

API

Currently, the following commands are available as members of the class SpotDriver which matches the API of DOGZILLA:

```
def forward(step)
def backward(step)
def turnleft(step)
def turnright(step)
def left(step)
def right(step)
def stop()
```

All these commands take in a step parameter which refers to the stride length of the robot. We also provide some commands to move over a specific amount of time, but this does not match the DOGZILLA API, so some caution will need to be taken when using it:

```
def move_forward(time)
def move_backward(time)
def turn_left(time)
def turn_right(time)
def move_forward_and_right(time)
def move_forward_and_left(time)
def move_backward_and_right(time)
def move_backward_and_left(time)
def stop_moving(time)
```

Finally, we also provide an API for grabbing the LIDAR sensor on Spot, providing an array of ranges similar to DOGZILLA:

```
def get_lidar_image()
```

5. Future Work

Improvements to Workflow and Simulation

It would be nice to have a model within Webots that matched DOGZILLA in scale and visually. A model would need to be created, ensuring it precisely matches the physical dimensions of DOGZILLA as well as the physical requirements of Spot. The controller would also need to be adjusted to accommodate the new dimensions, likely requiring new joint limits and leg lengths. An interface to unify the code running on DOGZILLA and Webots will also be needed, to ease porting from one platform to the other. A bash script may help with this. Additionally, a C++ port of the driver would likely help allow other controllers written in other languages to work. C++'s interoperability is much more expandable than attempting to embed Python in other languages. The main challenge that would come from this is the math the driver contains since it heavily relies on matrix algebra and the numpy library. There is a C++ port of numpy, but it has not been tested for our purposes.

Spring Semester

Starting in the spring semester, a focus on Spot and DOGZILLA's interoperability will be needed. This would primarily be around the aforementioned scale problem, but also through rigorous testing of code designed for Spot on DOGZILLA and vice versa. Getting comfortable with Webot's modeling system should also be prioritized in order to both create proper courses for Spot, as well as further attempting to re-create DOGZILLA virtually. A set of callipers and perhaps some sort of photogrammetry software may prove useful.

6. Resources and Directories

Github

- Github Login: **LocalRobots**
- Github Password: **Webots123()**
- Github Repository Link: <https://github.com/LocalRobots/SpotImplementation>
- Ideally any changes to this repository are pull requested and reviewed first before being merged to main.

7. References

<https://github.com/RRL-ALeRT/spot-webots-cloud/tree/main>

https://github.com/MASKOR/webots_ros2_spot

https://www.researchgate.net/publication/322594373_Inverse_Kinematic_Analysis_of_a_Quadraped_Robot

<https://cyberbotics.com/doc/guide/index>

<https://github.com/dpilger26/NumCpp>

<https://github.com/LucaVits/LearningROSWithPythonTurtle/blob/main/ROS2/WindowsInstallation/ROS2IronInstallationWindows.ipynb>